

Assignment 2

Zunaira Abdul Aziz
BSSE23058
Section-A

Q1:

1.

	Predicted Positive	Predicted Negative
Actual Positive	TP = 0	FN = 50
Actual Negative	FP = 0	TN = 9,950

2.

Accuracy = $TP + TN / \text{Total} = 0 + 9950 / 10000 = 0.995$ or 99.5%

Recall = $TP / TP + FN = 0 / 0 + 50 = 0$ or 0%

Accuracy is a fundamentally flawed metric for this dataset because the data is severely imbalanced, allowing a lazy model to achieve a 99.5% score simply by guessing the majority class the legitimate class every single time. Consequently, this high accuracy completely masks the fact that the model is entirely useless at its actual job: detecting fraudulent transactions.

3.

SMOTE's Geometric Mechanism: Instead of just duplicating existing data, SMOTE creates synthetic minority-class samples by operating directly in the feature space. It takes a minority data point, finds its k-nearest minority neighbors, draws imaginary line segments connecting them, and then randomly places new, artificial data points along those lines.

Contrast with Cost-Sensitive Learning: While SMOTE physically changes the dataset by adding new synthetic rows to balance the classes, Cost-Sensitive Learning leaves the dataset

exactly as it is. Instead, it modifies the loss function to heavily penalize the model for missing a minority class instance. In this scenario, missing a fraud would cost the model far more than misclassifying a legitimate transaction.

Practical Advantages and Limitations:

SMOTE

Advantage: It reduces the risk of overfitting compared to simple random oversampling because it creates diverse, brand-new examples rather than just copying old ones.

Limitation: If minority points are located near or inside the majority class territory (outliers), SMOTE can create overlapping synthetic noise that confuses the model. It also increases training time due to the larger dataset.

Cost-Sensitive Learning

Advantage: It is highly efficient because it doesn't increase the size of the dataset, keeping training times fast without introducing synthetic data points.

Limitation: It can be very difficult to figure out the optimal penalty weights to assign to the classes; getting it wrong requires a lot of trial and error and can drastically increase False Positives.

Q2:

1.

K-Means is fundamentally incapable of separating concentric rings due to its underlying geometry and objective function. K-Means tries to minimize the Within Cluster Sum of Squares, which is the sum of the squared Euclidean distances between every data point and the centroid of its assigned cluster.

Why this fails on rings:

1. **Assumption of Spherical Blobs:** Because the objective function relies strictly on measuring the straight-line distance to the centroid, K-Means strongly assumes that clusters are convex, spherical, and compact.
2. **The Centroid Problem:** The geometric center of a ring-shaped cluster is empty space right in the middle of the circle. If you have two nested rings, they actually share the exact same geometric center.
3. **The Result:** Instead of recognizing that points are connected in a continuous curve, K-Means will try to minimize distances by dropping centroids side-by-side and splitting the rings down the middle, slicing the data into wedges like a pie. It simply cannot draw curved boundaries to separate an inner shape from an outer shape.

2.

Single-linkage is the right choice for concentric rings. Here is the breakdown of why your teammate is incorrect, based on the definitions of the linkage criteria.

Definitions:

- **Single-Linkage (Nearest Neighbor):** The distance between two clusters is defined as the shortest distance between any single point in Cluster 1 and any single point in Cluster 2.
- **Complete-Linkage (Furthest Neighbor):** The distance between two clusters is defined as the maximum distance between any point in Cluster 1 and any point in Cluster 2.

Why Complete-Linkage Fails Here:

Complete-linkage wants all points inside a merged cluster to be relatively close to each other, meaning it strongly biases toward forming tightly packed, spherical blobs. If it tries to group a ring, the points on opposite sides of that ring are extremely far apart. The algorithm sees this massive furthest neighbor distance and penalizes it heavily. Instead of following the curve of the ring, it will break the rings apart into smaller, chunky clusters that look like little separate spheres.

Why Single-Linkage Succeeds:

Single-linkage excels at a phenomenon called chaining. Because it only cares about the shortest jump to the very next point, it will happily connect a point to its closest neighbor along the curve of the ring. As long as the points along the ring are close together, single-linkage will trace the entire circumference, perfectly separating the continuous inner ring from the continuous outer ring without caring about the massive distance between opposite sides of the circle.

Q3:

1. Identification of Data Leakage (5 marks)

The specific issue introduced here is Data Leakage (more specifically, a combination of Feature Distribution Leakage via the scaler and severe Target Leakage via the encoder).

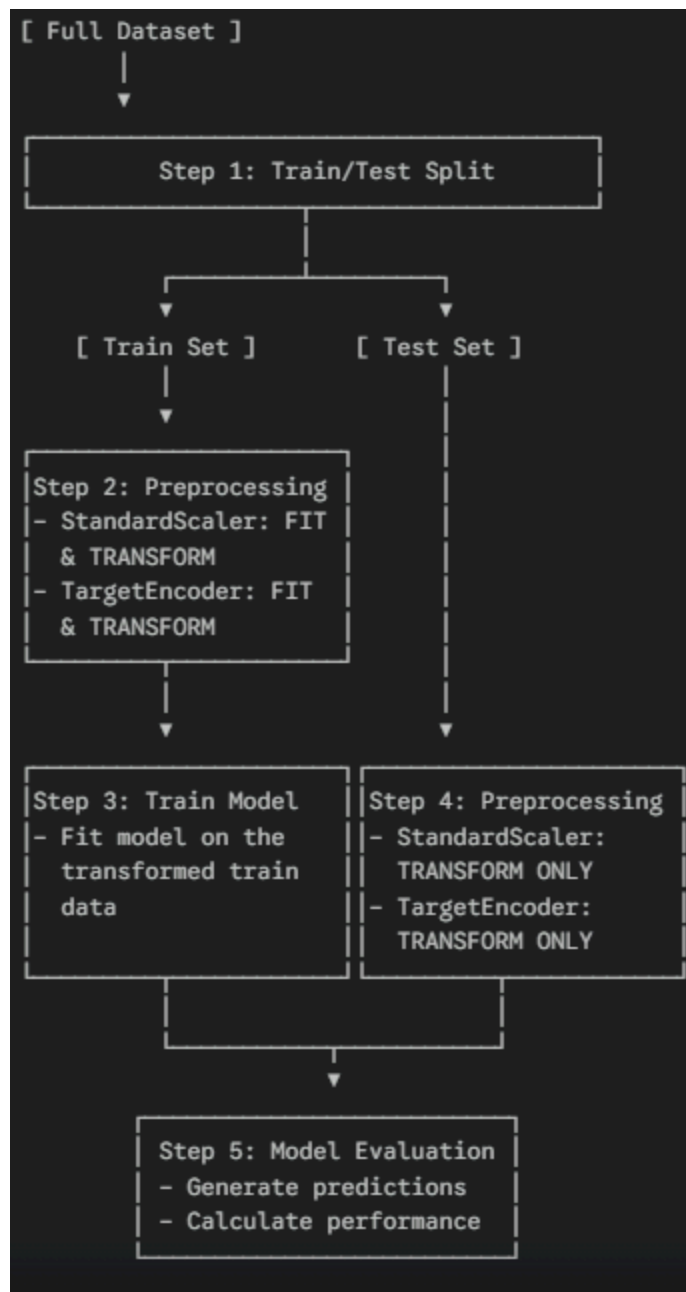
- **How it happens:** By fitting the transformers on the full dataset, the model gains look-ahead knowledge about the test set before the split even happens.
- **The Scaler Leakage:** The StandardScaler calculates the global mean and variance of

the full dataset. When applied to the training data, it incorporates information about the range and distribution of the test data.

- The Encoder Leakage: The TargetEncoder replaces categorical values with the expected mean of the target variable (exam scores). By fitting it on the entire dataset, the exact exam scores of the test set are mathematically blended into the training features. The model is essentially cheating by seeing the answers it is supposed to predict.
-

2.

To prevent leakage, the data must be split before any preprocessing parameters are calculated. Here is the correct workflow mapping out where transformers are fitted and applied versus where they are only applied:



Fitted vs. Applied Summary:

- Train Set: Both StandardScaler and TargetEncoder are Fitted and Applied (using `.fit_transform()`).
- Test Set: Both transformers are Only Applied (using `.transform()`) using the scaling and encoding rules learned exclusively from the training set.

3. Refuting the Teammate's Argument (5 marks)

The argument that a 20% leakage effect is negligible is dangerously incorrect for two major reasons:

1. **Overly Optimistic Metrics:** Target encoding is incredibly powerful because it builds a direct mathematical bridge to the target variable. Even if only 20% of the target data leaks, it will drastically and artificially inflate the model's test performance during development. Your team will believe you have a near-perfect model when, in reality, you don't.
2. **Catastrophic Failure in Production:** In the real world, future students will submit their study hours and departments *before* taking the exam. The model will no longer have a test set target to peek at. Because the model learned to rely heavily on the leaked target information during training, its performance will completely collapse when deployed on truly unseen, real-world data. Leakage doesn't just nudge the numbers; it fundamentally breaks the model's ability to generalize.

Q4:

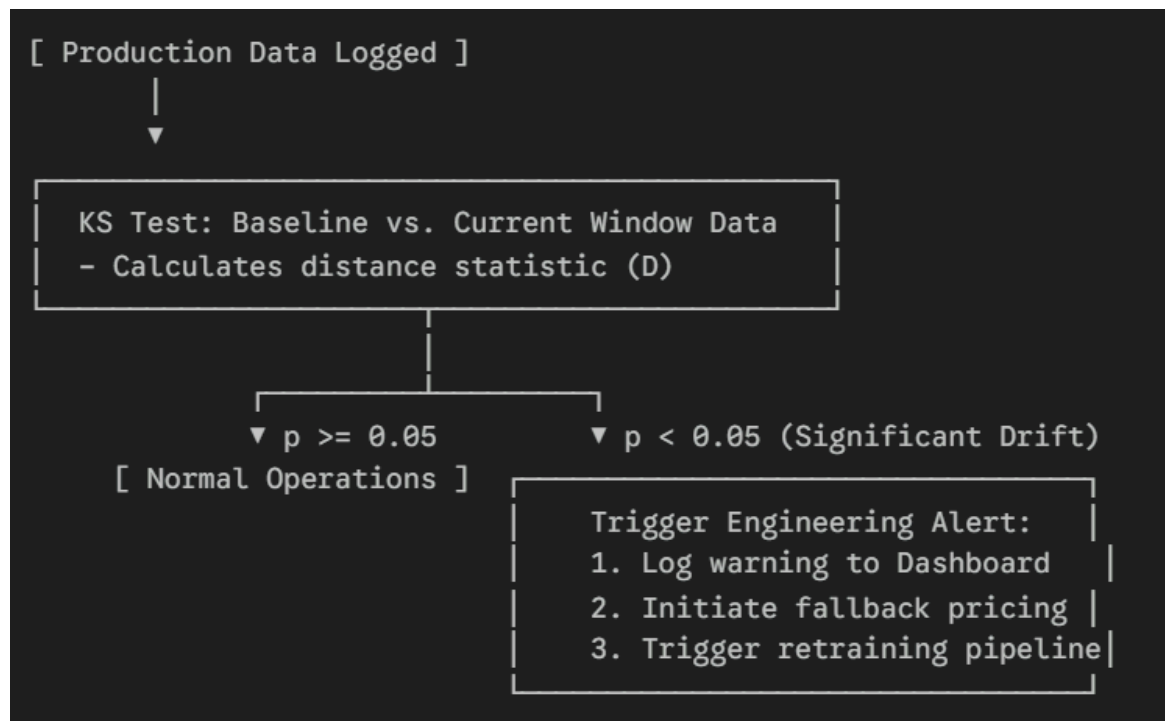
1.

When a model's performance degrades over time while the infrastructure remains perfectly intact, it is usually a sign of Model Drift. This can be categorized into two distinct phenomena:

- **Data Drift:** This occurs when the statistical distribution of the input features ($P(X)$) changes over time, but the underlying relationship between the features and the target variable remains exactly the same.
 - Ride-Sharing Example: A sudden, major influx of tourists or a massive tech conference moves into the city. The raw numbers for input features like current location passenger density or app opens per minute skyrocket to values the model has never seen before, even though a high density still means high demand.
- **Concept Drift:** This occurs when the statistical properties of the target variable change, meaning the underlying relationship between the input features and the output ($P(Y|X)$) has fundamentally shifted. The data looks the same, but the real-world meaning of that data has changed.
 - Ride-Sharing Example: A competitor enters the market and launches a massive discount campaign, or a city drastically expands its free public subway system. Even if weather, time of day, and historical demand features remain identical to previous months, the actual ride sharing demand will plummet because passenger behavior has shifted.

2.

The KS test can act as an early warning system in your production pipeline, flagging changes in numerical features before they completely ruin the model's predictions.



(a) The Two Distributions Compared: The test compares a Baseline Distribution (the distribution of a numerical feature, like surge multiplier or temperature, taken from the gold-standard training/validation set) against the Current Production Window Distribution (the distribution of that same feature collected from live API logs over the last 24 hours or week).

(b) What the Test Statistic Measures: The KS test statistic (denoted as D) measures the maximum vertical distance between the Cumulative Empirical Distribution Functions (CDFs) of the two samples. If the two shapes are nearly identical, D will be close to 0; if the production data has shifted or spread out, D will grow larger.

(c) The Engineering Response to a Significant Result ($p < 0.05$): A p -value below 0.05 indicates a less than 5% chance that the difference between production and training data happened by pure random coincidence. This should automatically trigger:

1. **Alerts:** An automated ticket (e.g. via PagerDuty/Slack) to the MLOps team warning of feature drift.
2. **Fallback Mechanisms:** Switching the system to a safer baseline or

heuristic-based pricing model if the drift is severe.

- 3. **Automation:** Kicking off an automated data collection and retraining pipeline to update the model with the newly shifted data.

3.

Red Teaming Method for Ride-Sharing: Before initial deployment, a machine learning team can perform Stress Testing / Boundary Red Teaming. This involves intentionally feeding the model extreme, adversarial edge-case scenarios to see if it breaks or produces dangerous predictions. For example, a red teamer might simulate a massive, impossible weather anomaly (e.g. 50°C temperatures during a simulated blizzard) combined with overlapping mega-events to see if the demand model outputs negative numbers, infinity values, or crashes the pricing API entirely.

Adversarial Inputs: Tabular vs. Image Domain An Adversarial Input in machine learning is a sample that has been deliberately modified with subtle changes to trick a model into making a confident, incorrect prediction.

The differences between tabular regression and image-domain examples are stark:

Attribute	Image-Domain Adversarial Examples	Tabular Regression Adversarial Examples
Nature of Perturbation	Pixel-level noise: Tiny, imperceptible changes are added across thousands of pixels. To a human, the image looks completely unaltered (e.g., a panda still looks exactly like a panda).	Feature-level manipulation: Specific, discrete column values are altered (e.g., tweaking "study hours" from 4 to 4.2, or changing "passenger rating").
Constraints & Logic	Low constraint: Pixels can easily be adjusted up or down freely within a standard 0 to 255 scale without breaking real-world logic.	High constraint / Hard Rules: Features must respect strict domain logic and data types. You cannot make "ride distance" a negative number, or change a categorical variable like City Zone to an impossible fractional value.

Impact of the Output	Misclassification: The goal is usually to flip a discrete categorical label (e.g. making a Stop Sign look like a Speed Limit sign to an autonomous vehicle).	Value Distortion: The goal is to maximize or minimize a continuous number (e.g. finding the exact combination of minor tweaks that tricks the model into making a ride price artificially cheap or dangerously expensive).
-----------------------------	---	---